

Unit 3 · Topic Notes

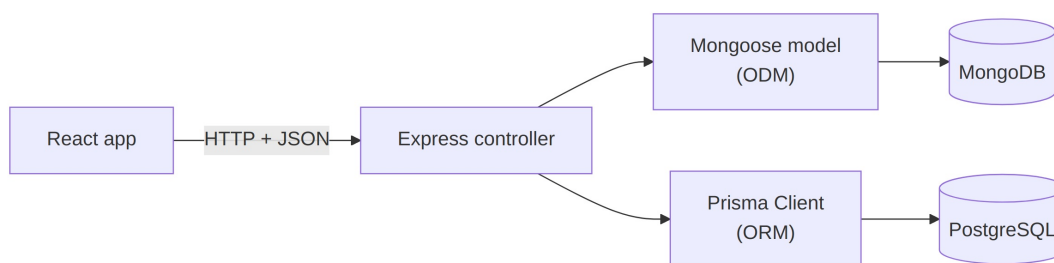
Database Setup with an ODM/ORM — MongoDB + Mongoose, PostgreSQL + Prisma

Course: Full Stack Application Development with MERN

Learning Objectives

1. Choose between MongoDB and PostgreSQL for a given project
2. Connect an Express app to MongoDB Atlas using Mongoose
3. Define schemas with types, validation, defaults, indexes and timestamps
4. Model relationships using references vs. embedding
5. Recognise the common connection and schema mistakes before they bite you

1. Where the Database Fits



Database layer diagram

An **ODM** (Object Document Mapper, e.g. Mongoose) maps JavaScript objects to MongoDB documents. An **ORM** (Object Relational Mapper, e.g. Prisma, Sequelize) maps objects to rows in SQL tables. Both give you validation, a cleaner query API, and protection from hand-written query strings.

2. MongoDB vs. PostgreSQL

	MongoDB (document)	PostgreSQL (relational)
Data shape	JSON-like documents, flexible	Tables with fixed columns
Schema	Optional (Mongoose adds one)	Enforced by the database
Relationships	References or embedding	Foreign keys + JOINS
Transactions	Supported, used less often	Core strength
Best for	Fast-changing shapes, content, catalogues	Money, inventory, strict integrity
Typical MERN choice	Default for this course	Good alternative, common in industry

Rule of thumb: if you keep drawing many-to-many relationships and need strict integrity (payments, bookings), lean SQL. If your data is document-shaped (a blog post with comments, a product with variants), MongoDB fits naturally.

3. MongoDB + Mongoose Setup

3.1 Install and configure

```
npm install express mongoose dotenv
```

Create a free cluster on MongoDB Atlas, create a database user, and copy the connection string into `.env` (never into your code):

```
# .env
PORT=5000
MONGO_URI=mongodb+srv://appuser:<password>@cluster0.abcde.mongodb.net/shopdb?retryWrites=true&w=majority
```

Add `.env` to `.gitignore` **before your first commit**.

3.2 Connection module

```
// config/db.js
import mongoose from "mongoose";

export async function connectDB() {
  try {
    const conn = await mongoose.connect(process.env.MONGO_URI);
    console.log(`MongoDB connected: ${conn.connection.host}`);
  } catch (err) {
    console.error("MongoDB connection failed:", err.message);
    process.exit(1); // no point running an API without its database
  }
}
```

```
// server.js
import "dotenv/config";
import express from "express";
import { connectDB } from "./config/db.js";

const app = express();
app.use(express.json());

await connectDB(); // connect first...
app.listen(process.env.PORT || 5000, () => // ...then accept requests
  console.log("Server running")
);
```

3.3 Connection edge cases

Symptom	Likely cause	Fix
<code>MongoServerSelectionError</code> / timeout	Your IP is not on the Atlas Network Access allow-list	Add your current IP (or <code>0.0.0.0/0</code> for learning only, never for production)
<code>bad auth : Authentication failed</code>	Wrong user/password, or password not URL-encoded	Special characters like <code>@</code> , <code>#</code> , <code>/</code> in the password must be encoded (<code>@</code> becomes <code>%40</code>)
<code>MONGO_URI</code> is <code>undefined</code>	<code>.env</code> not loaded, or <code>dotenv</code> imported after use	Put <code>import "dotenv/config"</code> on the first line of <code>server.js</code>
Works locally, fails on Render/Vercel	Env variable not set on the host, or Atlas blocks the host's IP	Add the variable in the host dashboard; allow-list the host
Connected to the wrong database	Database name missing from the URI (defaults to <code>test</code>)	Put the name after the host: <code>.../shopdb?...</code>

4. Defining Schemas and Models

```
// models/Product.js
import mongoose from "mongoose";

const productSchema = new mongoose.Schema(
  {
    name: {
      type: String,
      required: [true, "Product name is required"],
      trim: true,
      minlength: 2,
      maxlength: 100,
    },
    price: {
      type: Number,
      required: true,
      min: [0, "Price cannot be negative"],
    },
    stock: { type: Number, default: 0, min: 0 },
    category: {
      type: String,
      enum: { values: ["books", "stationery", "electronics"], message: "{VALUE} is not a valid category" },
    },
    sku: { type: String, unique: true, uppercase: true },
    isActive: { type: Boolean, default: true },
  },
  { timestamps: true } // adds createdAt and updatedAt automatically
);

// Indexes speed up the queries you run most often
productSchema.index({ category: 1, price: 1 });

// Virtual: computed on the fly, not stored
productSchema.virtual("inStock").get(function () {
  return this.stock > 0;
});

export default mongoose.model("Product", productSchema);
```

Mongoose turns the model name `Product` into the collection `products` (lower-cased and pluralised).

4.1 Schema edge cases

1. Validation errors look different from other errors.

```
try {
  await Product.create({ name: "A", price: -5 });
} catch (err) {
  console.log(err.name); // "ValidationError"
  console.log(Object.values(err.errors).map((e) => e.message));
}
```

2. `unique: true` is not a validator, it creates an index. Duplicates produce a MongoDB error with `code === 11000`, not a `ValidationError`:

```
try {
  await Product.create({ name: "Pen", price: 10, sku: "PEN-1" });
  await Product.create({ name: "Pen 2", price: 12, sku: "PEN-1" }); // duplicate
} catch (err) {
  if (err.code === 11000) console.log("Duplicate:", err.keyValue); // { sku: 'PEN-1' }
}
```

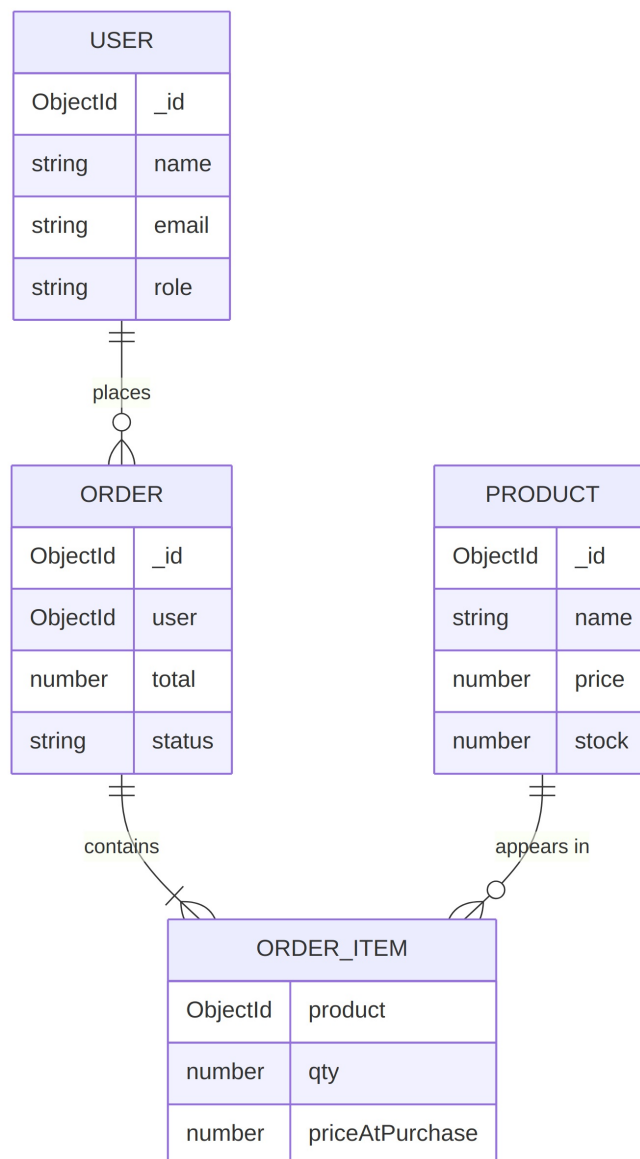
Also: if you add `unique` to a collection that already has duplicates, the index build fails silently at startup.

3. Extra fields are silently dropped. With the default `strict` mode, `Product.create({ name: "Pen", price: 5, hack: true })` saves without `hack`. That is a feature (it blocks unexpected fields), but it means a typo like `pirce` gives you a “price is required” error, not a saved bad field.

4. Type casting is generous. `price: "45"` is cast to `45`. But `price: "abc"` throws a `CastError`. And `{ price: {} }` also throws. Never assume the raw request body has the right types (see the validation topic).

5. `required` accepts `0` and `false`, but rejects `""`, `null` and `undefined`. A `stock` of `0` is fine.
6. Defaults are applied on create, not on update. `default: true` will not appear on documents inserted before you added the default.
7. Arrays default to `[]`. A field like `tags: [String]` is never `undefined`; it is an empty array.
-

5. Relationships: Reference vs. Embed



Entity relationship diagram

```
// models/Order.js
const orderItemSchema = new mongoose.Schema(
  {
    product: { type: mongoose.Schema.Types.ObjectId, ref: "Product", required: true },
    qty: { type: Number, required: true, min: 1 },
    priceAtPurchase: { type: Number, required: true }, // snapshot!
  },
  { _id: false }
);

const orderSchema = new mongoose.Schema(
  {
    user: { type: mongoose.Schema.Types.ObjectId, ref: "User", required: true },
    items: [orderItemSchema], // embedded: always read together with the order
    total: { type: Number, required: true },
    status: { type: String, enum: ["pending", "paid", "shipped"], default: "pending" },
  },
  { timestamps: true }
);

export default mongoose.model("Order", orderSchema);
```

Reading the referenced data with `populate` :

```
const order = await Order.findById(id)
  .populate("user", "name email") // only these fields
  .populate("items.product", "name price");
```

Embed when...	Reference when...
Data is always read together (order items)	Data is shared or large (users, products)
The child has no life of its own	The child is updated independently
The array stays small and bounded	The array can grow without limit (comments on a viral post)

Edge cases

- **Price snapshot:** store `priceAtPurchase` inside the order. If you only reference the product, changing its price rewrites history for every old order.
- **Deleted references:** if a product is deleted, `populate` returns `null` for that item. Handle `null` in your code or use soft delete (`isActive: false`).
- **MongoDB has no foreign key enforcement.** Nothing stops you saving an order with a `user` id that does not exist. Check in your controller.
- **The 16 MB document limit** makes unbounded embedded arrays a bug waiting to happen.

6. PostgreSQL + Prisma (the SQL alternative)

Prisma is a type-safe ORM. You describe your tables in `schema.prisma` , and Prisma generates migrations and a client.

```
// prisma/schema.prisma
model User {
  id      Int      @id @default(autoincrement())
  email   String   @unique
  name    String?
  orders  Order[]
}

model Order {
  id      Int      @id @default(autoincrement())
  total   Decimal  @db.Decimal(10, 2)
  status  String   @default("pending")
  createdAt DateTime @default(now())
  user    User     @relation(fields: [userId], references: [id])
  userId  Int
}
```

```
npx prisma migrate dev --name init # creates tables + a migration file
npx prisma studio                  # visual data browser
```

```
// Queries (prisma is your PrismaClient instance)
const user = await prisma.user.create({ data: { email: "asha@example.com", name: "Asha" } });

const orders = await prisma.order.findMany({
  where: { userId: user.id, status: "pending" },
  include: { user: true },
  orderBy: { createdAt: "desc" },
});
```

Version note: Prisma’s setup steps (config file, client generator and the import path of the generated client) changed between major versions. The schema and query code above stay the same, but for install and configuration always follow the current official quickstart at prisma.io/docs.

Edge cases

- Use `Decimal` for money, never `Float` (floating-point rounding errors).
- `findUnique` returns `null` when nothing matches; `findUniqueOrThrow` throws. Pick deliberately.
- A unique-constraint violation throws error code `P2002`, the Prisma equivalent of MongoDB’s `11000`.
- Never edit a migration that has already been applied; create a new one.

7. Hands-On Lab (45 min, individual)

1. Create a free MongoDB Atlas cluster, a database user, and allow-list your IP.
2. Build `config/db.js` and `server.js` as in §3.2. Confirm “MongoDB connected” prints.
3. Create the `Product` model from §4. Write a small script that creates three products.
4. Trigger each of these on purpose and note the error each produces: a negative price, a missing name, a duplicate `sku`, an invalid `category`.
5. Add an `Order` model with embedded items. Create one order and read it back with `populate`.
6. Deliberately break `MONGO_URI` (wrong password) and observe how your app behaves.

8. Quick Assessment

1. Name two differences between an ODM and an ORM.
2. Why must the app call `connectDB()` before `app.listen()`?
3. What error code does a duplicate `unique` field produce, and how is it different from a `ValidationError`?
4. Why store `priceAtPurchase` on the order instead of only referencing the product?
5. Your Atlas connection times out only on your college Wi-Fi. What do you check first?
6. When would you embed a document instead of referencing it?

9. Common Mistakes to Avoid

- Committing `.env` or pasting the connection string into source code
 - Forgetting the database name in the URI, so data lands in `test`
 - Treating `unique: true` as validation
 - Embedding an array that can grow forever
 - Using `Float` for money in SQL, or `Number` maths for currency without rounding
-

Notes prepared for Full Stack Learning — MERN course, Unit 3.